



Relatório do Trabalho Final
Introdução a Machine Learning

**Otimização de Farming em RPGs com
Aprendizado de Máquina: Um Sistema de
Recomendação para Maximização no Ganho
De Experiência em Pokémon**

Autores:

Giovanna Oliveira da Silva
Kayo Bacelar de Oliveira
Rodrigo da Silva Nogueira
Leonardo Ambrus de Lima
Isabella Vitoria da Silva Raymundo

7 de junho de 2026

Resumo

Este trabalho apresenta o desenvolvimento de um pipeline completo de Aprendizado de Máquina voltado à modelagem preditiva de combates e à formulação de um sistema de recomendação focado na otimização do ganho de pontos de Experiência (XP). Tomando como base o ecossistema competitivo da franquia Pokémon, o projeto utiliza algoritmos de classificação supervisionada para identificar oponentes que combinem vulnerabilidade matemática e alta rentabilidade de treino. A base de dados foi gerada a partir de um motor de batalhas determinístico em nível máximo (Level 100), consolidando um dataset sintético contendo 10.000 confrontos. O pipeline abrange desde o pré-processamento estruturado e engenharia de atributos — transformando relações elementares de tipos em features contínuas — até o ajuste fino de hiperparâmetros por validação cruzada exaustiva via *GridSearchCV*. Três arquiteturas preditivas foram avaliadas e contrastadas: Regressão Logística, *Random Forest* e *Multi-Layer Perceptron* (MLP). Como produto final, implementou-se um algoritmo recomendador para "*Farming Seguro*", capaz de filtrar adversários e ranqueá-los de modo a garantir uma probabilidade de vitória superior a 85%, articulando conceitos práticos de viés algorítmico, eficiência e capacidade de generalização.

Palavras-chave: Aprendizado de Máquina, Sistema de Recomendação, Regressão Logística, Pokémon, Otimização de Processos.

1 Introdução

Nos jogos de interpretação de papéis (*Role-Playing Games* - RPGs), o progresso de um personagem está intrinsecamente atrelado à aquisição contínua de pontos de Experiência (XP). Esse processo de acumulação, popularmente denominado "*farming*" ou "*grinding*", frequentemente exige que o jogador enfrente uma sequência repetitiva de combates para elevar os atributos de suas unidades. Na franquia Pokémon, esse cenário ganha contornos complexos devido ao vasto arranjo de variáveis envolvidas nas dinâmicas de combate, tais como vantagens elementares de tipos, distribuição de estatísticas base, golpes disponíveis e níveis dos oponentes. Maximizar a eficiência dessa rotina consiste em encontrar oponentes que cedam grandes quantidades de experiência, mas que minimizem o risco de derrota ou o desgaste excessivo da equipe do jogador.

Diante desse cenário, este trabalho propõe a aplicação de técnicas de Aprendizado de Máquina Supervisionado para mapear e mitigar os riscos associados ao *farming*. O objetivo central é estruturar um ecossistema inteligente de dados e modelagem preditiva capaz de prever o desfecho de batalhas e atuar ativamente como um sistema de recomendação de alvos ótimos para o treinamento de personagens específicos.

A abordagem metodológica compreende a engenharia de um motor de batalhas determinístico, parametrizado sob as regras de combates competitivos de nível máximo (Level 100), utilizado para confeccionar uma base de dados robusta contendo 10.000 simulações de confrontos. A partir deste repositório sintético, o pipeline de Ciência de Dados executa o tratamento de atributos categóricos e a conversão de multiplicadores de vantagens elementares em variáveis contínuas adequadas para o consumo de modelos estatísticos.

Com a finalidade de estabelecer contrastes arquitetônicos e avaliar o impacto da complexidade dos modelos na tarefa de classificação binária (Vitória versus Derrota), o projeto implementa e compara três classificadores distintos:

- **Regressão Logística:** Utilizada como *baseline* linear para quantificar a separabilidade básica do espaço amostral;
- **Random Forest:** Abordagem baseada em comitês de árvores de decisão para capturar interações não lineares e regras de corte abruptas nas features;
- **Multi-Layer Perceptron (MLP):** Arquitetura de rede neural artificial para ava-

liar a aproximação de fronteiras complexas de decisão através de camadas ocultas.

Os parâmetros de cada modelo são otimizados de forma rigorosa por meio de *Grid-SearchCV* acoplado a esquemas de validação cruzada. Por fim, as saídas probabilísticas calculadas pelos modelos servem de fundação para o desenvolvimento de um algoritmo recomendador focado em "*Farming Seguro*". Este mecanismo filtra e ranqueia potenciais oponentes, exibindo ao usuário apenas os alvos com probabilidade estimada de sucesso superior a 85%, traduzindo os conceitos de generalização matemática e análise de viés algorítmico discutidos no âmbito acadêmico em uma ferramenta aplicada de tomada de decisão.

2 Ferramentas e Justificativas

A construção do ecossistema computacional deste projeto foi pautada na escolha de tecnologias consagradas no mercado e na academia, visando garantir reprodutibilidade, eficiência no tratamento de dados e uma experiência de uso fluida. Esta seção detalha as ferramentas utilizadas, divididas entre o ambiente de processamento e modelagem e a infraestrutura de serviços e interface.

2.1 Scripts Executados e Modelagem de Aprendizado de Máquina

O núcleo de processamento de dados e inteligência preditiva foi desenvolvido inteiramente na linguagem *Python* (versão 3.12), utilizando um conjunto de scripts modulares estruturados para compor o pipeline de Ciência de Dados:

- **Processamento e Limpeza de Dados (`dados_processados.py`):** Utilizou-se a biblioteca *Pandas* para manipulação de dados tabulares. Esta etapa foi indispensável para o tratamento do *dataset* bruto do banco de dados Pokémon, extraindo numericamente os identificadores de *Base Exp* e *Catch Rate* via expressões regulares, preenchendo valores ausentes com médias amostrais e segmentando strings complexas de tipos elementares em variáveis normalizadas.
- **Geração de Dados Sintéticos (`simulador.py` e `utils.py`):** Devido à ausência de históricos públicos de confrontos detalhados, foi construído um motor de

simulação matemática determinístico. Baseando-se estritamente nos *status* máximos das criaturas e na fórmula oficial de cálculo de dano adaptada para o nível máximo (Level 100), o simulador gerou 10.000 registros de batalhas. A biblioteca *tqdm* foi integrada para monitoramento em tempo real do progresso da simulação. Os multiplicadores elementares de eficácia foram mapeados em matrizes associativas (dicionários nativos) para computar variáveis contínuas cruciais de vantagem de tipo.

- **Pipeline de Modelagem Prática (`pipeline_ml.py`):** O ecossistema de aprendizado foi estruturado utilizando a biblioteca *Scikit-Learn*. O script realiza a partição do *dataset* simétrico em subconjuntos de treino (80%) e teste (20%) através do *train_test_split*. Para o pré-processamento exigido por arquiteturas baseadas em gradiente e distância, incorporou-se o *StandardScaler* para a padronização Z-score das características. A otimização de hiperparâmetros utilizou a técnica exaustiva de *GridSearchCV* operando com validação cruzada em 3 *folds* ($cv = 3$) para mitigar riscos de sobreajuste (*overfitting*) no modelo de *Random Forest*.
- **Persistência de Modelos e Avaliação (`executar_avaliacao.py`):** Os modelos validados foram serializados em disco utilizando o módulo *pickle* (gerando arquivos `.pkl`), permitindo que os coeficientes calibrados da Regressão Logística, as estruturas de decisão do *Random Forest* e os pesos sinápticos da rede neural *Multi-Layer Perceptron* (MLP) fossem reutilizados instantaneamente sem necessidade de retreinamento. Um *wrapper* automatizado foi desenvolvido via módulo *subprocess* para disparar rotinas isoladas de avaliação de métricas.

2.2 Infraestrutura de Backend e Interface de Frontend

Para transformar as saídas probabilísticas dos modelos preditivos em uma aplicação útil para o usuário final, foi projetada uma arquitetura desacoplada cliente-servidor (API REST), baseada no documento guia do sistema:

- **Backend e Disponibilização de Serviços (`app.py`):** Desenvolvido utilizando o microframework *Flask*. A escolha do Flask justifica-se por sua leveza e excelente integração com o ecossistema de *Data Science* em Python. A API expõe a rota `/battle/matchups` utilizando o método HTTP POST, recebendo via formato

JSON o Pokémon desafiante e a tolerância de risco desejada pelo usuário. O servidor carrega os arquivos binários dos modelos em memória e calcula em tempo real as probabilidades de vitória aplicando o método `predict_proba`. A extensão *Flask-CORS* foi acoplada para contornar restrições de segurança de compartilhamento de recursos de origens cruzadas (*Cross-Origin Resource Sharing*), permitindo a comunicação transparente com o servidor cliente.

- **Frontend e Sistema de Recomendação Interactiva:** Construído em *React 18* utilizando a base de criação *Create React App* (*react-scripts* 5). O framework baseado em componentes foi escolhido para viabilizar uma interface reativa e performática. As requisições assíncronas direcionadas ao backend Flask são gerenciadas pela biblioteca *Axios*. O fluxo operacional da interface consome a API REST para capturar o array de predições, ordená-lo por taxa de sucesso decrescente e renderizar os cards dinâmicos dos oponentes. O comportamento dinâmico é integrado a seletores de limite (*sliders*), filtrando os alvos de acordo com o threshold estipulado pelo jogador para garantir as diretrizes estabelecidas de "*Farming Seguro*".

3 Fundamentos Teóricos

Esta seção expõe a fundamentação matemática e estatística dos três algoritmos de aprendizado supervisionado avaliados no pipeline produtivo. As definições seguem os conceitos formais abordados na literatura e nos materiais de suporte da disciplina.

3.1 Regressão Logística

A Regressão Logística é o ponto de partida natural para problemas de classificação binária onde a variável resposta assume valores discretos $y \in \{0, 1\}$. Diferente da regressão linear pura, cujo espaço de hipóteses mapeia saídas no intervalo $[-\infty, +\infty]$, o modelo logístico restringe suas predições ao intervalo linear $[0, 1]$, permitindo sua interpretação direta como uma probabilidade condicional $\pi(\mathbf{x}) = P(y = 1|\mathbf{x})$.

O modelo reaproveita a estrutura do escore linear $z = \mathbf{x}^\top \boldsymbol{\theta}$ e o submete a uma transformação não linear através da função logística cumulativa, ou função *sigmoid*, definida matematicamente como:

$$\pi(\mathbf{x}) = s(\mathbf{x}^\top \boldsymbol{\theta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\theta}}} \quad (1)$$

Onde $\boldsymbol{\theta}$ representa o vetor de parâmetros (pesos e bias) a ser calibrado. Como essa formulação não admite uma solução analítica fechada via Mínimos Quadrados Ordinários (OLS), a calibração dos parâmetros é obtida maximizando a função de *Likelihood* de uma distribuição de Bernoulli, o que equivale a minimizar a função de custo convexa denominada *Log-Loss* ou Entropia Cruzada Binária:

$$L(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log(\pi(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - \pi(\mathbf{x}^{(i)}))] \quad (2)$$

A otimização de $L(\boldsymbol{\theta})$ é executada iterativamente por meio do algoritmo do Gradiente Descendente (GD), atualizando-se os pesos na direção oposta ao vetor gradiente analítico:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \frac{1}{n} \mathbf{X}^\top (\boldsymbol{\pi} - \mathbf{y}) \quad (3)$$

3.2 *Random Forest*

O classificador *Random Forest* (Floresta Aleatória) baseia-se no princípio de aprendizado por comitês (*Ensemble Learning*), combinando a predição de múltiplas Árvores de Decisão (estruturas do tipo CART) estruturadas de forma independente. O objetivo central desta arquitetura é mitigar o principal ponto fraco das árvores individuais: a alta variância e a propensão ao sobreajuste (*overfitting*).

A construção de cada árvore baseia-se em dois pilares de aleatoriedade:

1. ***Bagging* (Bootstrap Aggregation):** Cada árvore da floresta é treinada com uma subamostra distinta dos dados originais, gerada por amostragem aleatória com reposição.
2. **Seleção Aleatória de Atributos (*Feature Subspace*):** Durante a divisão (*split*) de cada nó da árvore, o algoritmo não busca a melhor variável preditora entre todas as disponíveis no *dataset*, mas restringe a busca a um subconjunto aleatório de tamanho pré-fixado m (frequentemente parametrizado como $m = \sqrt{p}$, onde p é o total de atributos).

O critério de partição dos nós para problemas de classificação baseia-se na minimização da impureza das folhas, comumente mensurada através do Índice de Gini ou da Entropia.

O Índice de Gini para um nó m é expresso por:

$$H(X_m) = \sum_{k=1}^K p_{mk}(1 - p_{mk}) \quad (4)$$

Onde p_{mk} representa a proporção de instâncias da classe k no nó m . A predição final da floresta para uma nova instância de teste é consolidada por meio de votação por maioria majoritária (*majority voting*) entre todas as árvores individuais da estrutura.

3.3 Redes Neurais Artificiais (*Multi-Layer Perceptron* - MLP)

A arquitetura *Multi-Layer Perceptron* (MLP) é uma rede neural artificial acíclica direcionada (*Feedforward*), composta por uma camada de entrada, uma ou mais camadas ocultas (*hidden layers*) intermediárias e uma camada de saída. Cada camada é constituída por unidades de processamento elementares denominadas neurônios artificiais ou nós.

A computação em um neurônio isolado consiste em aplicar uma transformação linear sobre as saídas da camada anterior, ponderada por um vetor de pesos $\mathbf{w}$ e acrescida de um termo de *bias* b , seguida pela aplicação de uma função de ativação não linear $g(\cdot)$. Este processo impede que a rede colapse em um modelo linear equivalente, conferindo-lhe a capacidade de aproximar fronteiras de decisão altamente complexas. O fluxo de propagação para frente (*Forward Propagation*) em um neurônio j pertencente à camada l é descrito como:

$$z_j^{[l]} = \sum_i w_{ji}^{[l]} a_i^{[l-1]} + b_j^{[l]} \quad (5)$$

$$a_j^{[l]} = g(z_j^{[l]}) \quad (6)$$

No contexto deste projeto, utilizou-se a função de ativação Unidade Linear Retificada (*ReLU*), definida como $g(z) = \max(0, z)$, nas camadas ocultas para mitigar problemas relacionados ao desvanecimento do gradiente. Na camada final de saída, a função *sigmoid* é empregada para mapear a resposta no intervalo probabilístico $[0, 1]$.

O ajuste dos pesos sinápticos ocorre minimizando a função de perda através do algoritmo de Retropropagação (*Backpropagation*). Este método aplica a Regra da Cadeia do cálculo diferencial para computar as derivadas parciais da função de custo em relação a cada peso e *bias* da rede, propagando os erros da camada de saída de volta até a camada de entrada para alimentar otimizadores baseados em gradiente estocástico.